

COMPUTER NETWORKS PROJECT

Private Network Service Platform

Phase 1 Implementation Report

Infrastructure: Two Physical Macs with Combined Roles

Course	Computer Networks
Project Phase	Phase 1
Infrastructure	Type 2 – Two Physical Macs
Operating System	macOS
DNS	dnsmasq
Reverse Proxy	Nginx
Backends	Python HTTP Server
Security	TLS / HTTPS

Team Members

Arun Kumar Giri	2401010099
Gaurav Meena	2401010169
Ritik Ranjan	2401010388

October 2026

Contents

1	Introduction	3
2	Objectives	3
3	Team and Responsibilities	3
4	System Architecture	3
4.1	Machine Configuration	4
4.2	Architecture Diagram	4
5	Request Flow	4
6	Network Configuration	5
7	DNS Configuration	5
7.1	DNS Verification	6
8	Backend Services	6
8.1	Backend A	6
8.2	Backend B	7
8.3	Backend Status API	7
9	Nginx Reverse Proxy and Load Balancer	7
9.1	Reverse Proxy Behavior	8
10	Round-Robin Load Balancing Verification	8
11	TLS and HTTPS	8
11.1	HTTPS Configuration	9
11.2	HTTPS Verification	9
12	HTTP Caching	9
13	Testing and Validation	10
13.1	Backend A Validation	10
13.2	Backend B Validation	10
13.3	Nginx Validation	10
13.4	End-to-End Validation	11
14	Wireshark Packet Analysis	11
15	Security Considerations	11
16	Project Structure	12
17	Phase 1 Completion Checklist	13
18	Conclusion	13

19 References**13**

1 Introduction

This project implements a private network service platform using two physical macOS machines connected through a private LAN.

The Phase 1 implementation establishes a complete application request path consisting of local DNS resolution, HTTPS termination, reverse proxying, round-robin load balancing, and two independent backend services.

No public cloud infrastructure is required for the implementation.

The architecture combines multiple infrastructure roles because only two physical Macs are available.

2 Objectives

The objectives of Phase 1 are:

- Establish private LAN communication between two Macs.
- Configure a local DNS service using dnsmasq.
- Resolve the application hostname to the Nginx entry point.
- Deploy Backend A on port 3001.
- Deploy Backend B on port 3002.
- Configure Nginx as a reverse proxy.
- Configure round-robin load balancing between the two backends.
- Configure TLS termination at Nginx.
- Provide HTTP caching headers.
- Verify the complete DNS-to-HTTPS-to-backend request path.
- Prepare evidence for DNS, TCP, TLS, HTTP, caching, and load balancing.

3 Team and Responsibilities

Name	Roll Number	Responsibility
Arun Kumar Giri	2401010099	Mac B – Nginx HTTPS + Backend B
Gaurav Meena	2401010169	Mac A – DNS + Backend A
Ritik Ranjan	24010100mac2	Documentation / Network Analysis

Table 1: Project Team

4 System Architecture

The Phase 1 system uses two physical macOS machines with combined roles.

4.1 Machine Configuration

Machine	LAN IP	Services	Role
Mac A	10.7.6.85	dnsmasq :53, Backend A :3001	DNS + Backend A
Mac B	10.7.0.248	Nginx :443, Backend B :3002	HTTPS + LB + Backend B

Table 2: Phase 1 Machine Configuration

4.2 Architecture Diagram

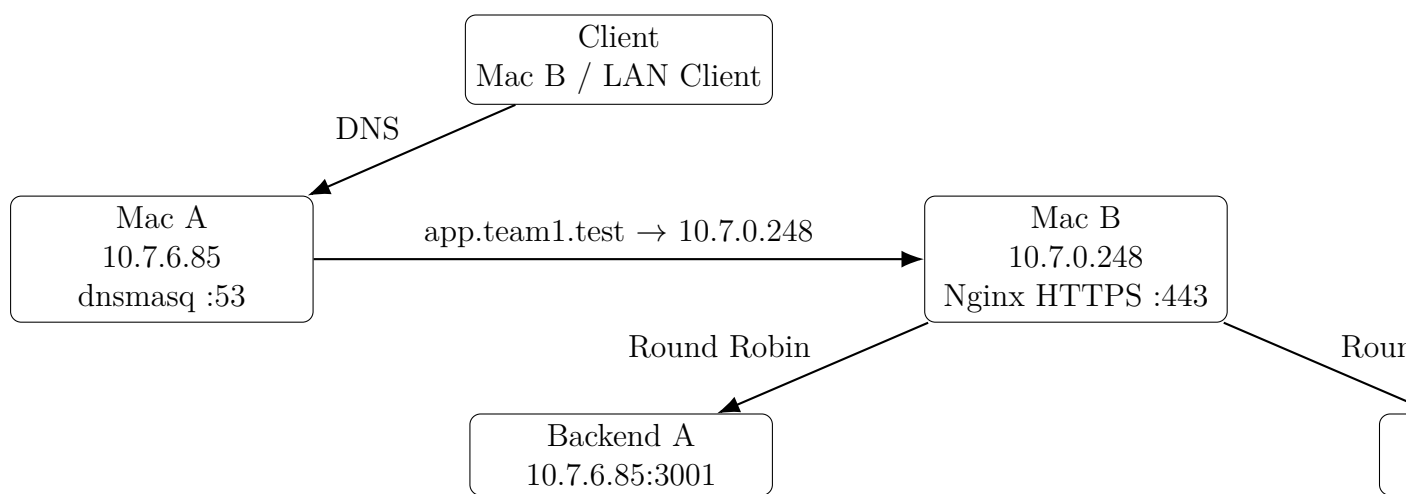


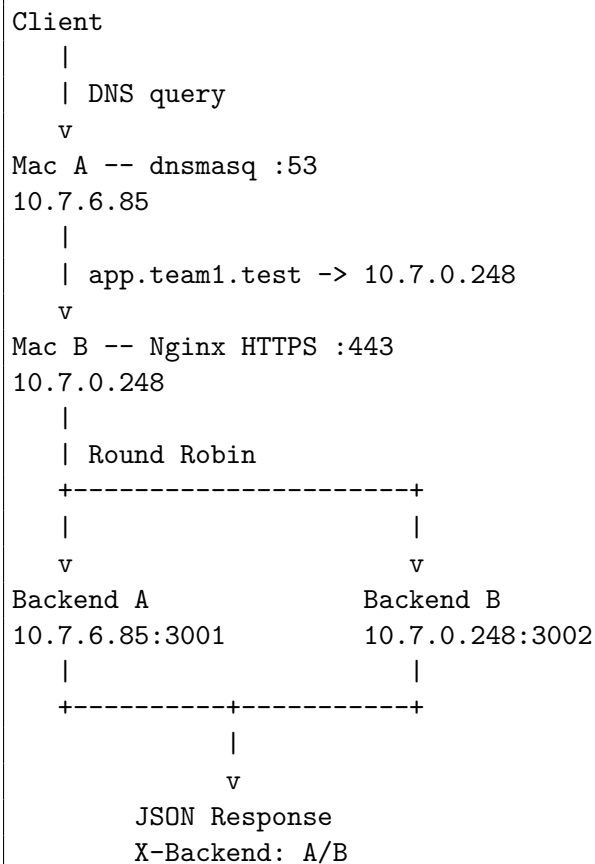
Figure 1: Two-Mac Phase 1 Architecture

5 Request Flow

The complete request flow is:

1. The client requests the IP address for `app.team1.test`.
2. The DNS query is sent to dnsmasq running on Mac A at 10.7.6.85:53.
3. dnsmasq returns the Nginx address 10.7.0.248.
4. The client establishes a TCP connection to Nginx on TCP port 443.
5. TLS is negotiated between the client and Nginx.
6. Nginx terminates TLS and forwards the HTTP request to the backend pool.
7. The upstream pool contains Backend A and Backend B.
8. Nginx distributes requests using round-robin load balancing.
9. The selected backend returns a JSON response containing its backend identifier.

10. Nginx returns the response to the client.



6 Network Configuration

The two machines were verified to be reachable over the private network using ICMP ping.

Component	Value	Purpose
Mac A	10.7.6.85	DNS + Backend A
Mac B	10.7.0.248	Nginx + Backend B
Backend A	TCP 3001	Backend service A
Backend B	TCP 3002	Backend service B
DNS	TCP/UDP 53	Hostname resolution
HTTPS	TCP 443	Secure application entry point

Table 3: Network Parameters

7 DNS Configuration

The local DNS service was implemented using dnsmasq on Mac A.

The project hostname is:

`app.team1.test`

The DNS record points to the Nginx public entry point rather than directly to either backend.

Hostname	Resolved IP	Target
<code>app.team1.test</code>	<code>10.7.0.248</code>	Nginx on Mac B

Table 4: DNS Configuration

7.1 DNS Verification

DNS was first verified directly against Mac A:

```
dig @10.7.6.85 app.team1.test
```

The observed response contained:

```
status: NOERROR

ANSWER SECTION:
app.team1.test.      A      10.7.0.248

SERVER:
10.7.6.85#53
```

This demonstrates that Mac B can reach the DNS service running on Mac A and resolve the application hostname.

8 Backend Services

Two independent HTTP backend services were implemented using Python's built-in HTTP server framework.

8.1 Backend A

Backend A runs on Mac A at:

`10.7.6.85:3001`

The service exposes:

- `/`
- `/api/status`
- HTTP GET
- HTTP HEAD

- X-Backend: A
- Cache-Control: max-age=60
- ETag

8.2 Backend B

Backend B runs on Mac B at:

10.7.0.248:3002

The service exposes:

- /
- /api/status
- HTTP GET
- HTTP HEAD
- X-Backend: B
- Cache-Control: max-age=60
- ETag

8.3 Backend Status API

The status endpoint is:

```
/api/status
```

Backend A returns a response identifying itself as backend A, while Backend B returns a response identifying itself as backend B.

9 Nginx Reverse Proxy and Load Balancer

Nginx is deployed on Mac B and acts as the single public application entry point.

The upstream backend pool is:

```
upstream backend_pool {  
    server 10.7.6.85:3001;  
    server 10.7.0.248:3002;  
}
```

The Nginx server listens for HTTPS traffic on TCP port 443.

Nginx forwards incoming requests to the backend pool using round-robin load balancing.

9.1 Reverse Proxy Behavior

The client does not need to directly access Backend A or Backend B during normal application requests.

Instead:

```
Client
  |
  v
Nginx :443
  |
  +----> Backend A :3001
  |
  +----> Backend B :3002
```

10 Round-Robin Load Balancing Verification

Multiple requests were sent through the Nginx endpoint.

The observed sequence was:

Request	Backend	Port
1	B	3002
2	A	3001
3	B	3002
4	A	3001
5	B	3002
6	A	3001

Table 5: Observed Nginx Round-Robin Sequence

The response body and X-Backend header were used to identify the backend that handled each request.

Example:

```
X-Backend: B
{"backend": "B", "status": "ok", "port": 3002}

X-Backend: A
{"backend": "A", "status": "ok", "port": 3001}
```

11 TLS and HTTPS

TLS termination is implemented at Nginx on Mac B.

The application hostname is:

`app.team1.test`

A self-signed X.509 certificate was generated using OpenSSL.
The certificate contains:

```
CN = app.team1.test  
  
Subject Alternative Name:  
DNS:app.team1.test  
IP Address:10.7.0.248
```

The certificate and private key are stored locally as:

```
ssl/cert.pem  
ssl/key.pem
```

The certificate validity period is one year.

11.1 HTTPS Configuration

Nginx terminates TLS on:

`10.7.0.248:443`

The configured TLS protocols are TLS 1.2 and TLS 1.3.

11.2 HTTPS Verification

HTTPS was tested using:

```
curl -I \  
  --resolve app.team1.test:443:10.7.0.248 \  
  --cacert ssl/cert.pem \  
  https://app.team1.test/api/status
```

The verified response was:

```
HTTP/1.1 200 OK  
Server: nginx/1.31.6  
Content-Type: application/json  
Cache-Control: max-age=60  
ETag: 17317b305d10a70607895d89245f1927  
X-Backend: A
```

The test did not use the `-k` option, so certificate validation was not bypassed.

12 HTTP Caching

The backend services provide HTTP caching metadata.

The primary caching directive is:

```
Cache-Control: max-age=60
```

An ETag value is also generated for the response.

Example:

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=60
ETag: 7670204eee0f572980a8aa672eb6ba50
X-Backend: B
```

Caching headers were verified through HTTP HEAD requests and through the Nginx HTTPS endpoint.

13 Testing and Validation

13.1 Backend A Validation

Backend A was tested locally and from Mac B.

The following functionality was verified:

- Root endpoint responds successfully.
- `/api/status` returns Backend A status.
- `X-Backend: A` is present.
- `Cache-Control: max-age=60` is present.
- ETag is present.

13.2 Backend B Validation

Backend B was tested locally on Mac B.

The following functionality was verified:

- Root endpoint responds successfully.
- `/api/status` returns Backend B status.
- `X-Backend: B` is present.
- `Cache-Control: max-age=60` is present.
- ETag is present.

13.3 Nginx Validation

Nginx configuration validation was performed using:

```
nginx -t
```

The configuration returned:

```
syntax is ok
test is successful
```

Nginx was verified to listen on TCP port 443.

13.4 End-to-End Validation

The final application path was validated as:

```
app.team1.test
  |
  v
10.7.0.248:443
  |
  v
Nginx
  |
+-----> Backend A :3001
  |
+-----> Backend B :3002
```

14 Wireshark Packet Analysis

Wireshark analysis is part of the Phase 1 network evidence.

The capture should demonstrate the following protocol sequence:

1. DNS query and DNS response.
2. TCP connection establishment.
3. TLS handshake.
4. HTTPS application traffic.
5. HTTP response metadata.
6. Communication between Nginx and the selected backend.

The evidence directory is organized as:

```
evidence/
  dns/
  tcp/
  tls/
  http/
  load-balancing/
  caching/
```

15 Security Considerations

The following security practices were followed:

- TLS is used for the public application entry point.
- The private key is stored locally.

- Certificate validation is performed without using `curl -k` for the final HTTPS test.
- Backend services are accessed through the Nginx entry point during normal application traffic.
- No passwords or application secrets are stored in source code.

16 Project Structure

The Phase 1 project is organized as follows:

```
CN-Project1/
|
+-- README.md
|
+-- backend-a/
|   +-- server.py
|
+-- backend-b/
|   +-- server.py
|
+-- nginx/
|   +-- nginx.conf
|
+-- ssl/
|   +-- cert.pem
|   +-- key.pem
|
+-- scripts/
|   +-- start-backend-b.sh
|   +-- stop-backend-b.sh
|   +-- test-backend-b.sh
|   +-- test-load-balancer.sh
|   +-- network-info.sh
|
+-- evidence/
|   +-- dns/
|   +-- tcp/
|   +-- tls/
|   +-- http/
|   +-- load-balancing/
|   +-- caching/
|
+-- docs/
|   +-- topology.md
|   +-- ip-addresses.md
|   +-- request-flow.md
|
+-- latex/
    +-- main.tex
```

17 Phase 1 Completion Checklist

Requirement	Status
Private LAN connectivity	✓
DNS service	✓
DNS record resolution	✓
Backend A :3001	✓
Backend B :3002	✓
Nginx reverse proxy	✓
Round-robin load balancing	✓
TLS certificate	✓
HTTPS :443	✓
Certificate validation without -k	✓
Cache-Control	✓
ETag	✓
Wireshark evidence	Pending / To be attached
Final documentation	✓

Table 6: Phase 1 Completion Status

18 Conclusion

Phase 1 successfully establishes a private network service platform using two physical macOS machines.

Mac A provides local DNS resolution and Backend A, while Mac B provides the Nginx HTTPS entry point and Backend B.

The hostname `app.team1.test` resolves to the Nginx server. Nginx terminates TLS on port 443 and distributes application requests between the two backend services using round-robin load balancing.

The backend responses provide explicit backend identification through the `X-Backend` header. HTTP caching metadata is provided using `Cache-Control: max-age=60` and ETag values.

The implementation therefore establishes the main Phase 1 networking path from DNS resolution through HTTPS and load-balanced backend services.

19 References

1. Computer Networks Project Specification, “Private Network Service Platform,” Phase 1.
2. Nginx reverse proxy and upstream load-balancing configuration used in the project.

3. Python HTTP backend implementation used for Backend A and Backend B.
4. dnsmasq local DNS configuration used on Mac A.
5. OpenSSL-generated TLS certificate used for `app.team1.test`.